

Practicum No.7**Date:.....****Cache Mapping Technique Analysis****I. Practical Significance**

Cache memory plays a critical role in speeding up data access in computers. Different cache mapping techniques like direct, associative, and set-associative affect how data is stored and retrieved. By analyzing these methods with the given mapping approach, students can understand the trade-offs between speed and efficiency. This practicum helps students appreciate how smart memory use can improve system performance.

II. Competency and Industry-Specific Practical Skills

This practicum exercise is expected to develop the following skills for the industry-identified competency.

- **Evaluate** the performance of scalable architectures for intelligent systems to ensure reliability.
(As stated in the Course Structure)
- a. **Implement** cache mapping techniques and simulate direct memory access operations. (PDO)

III. Relevant CO(s) (As stated in the Course Structure)

- CO3. **Interpret** memory functions and techniques influencing performance.

IV. Practical Outcome (PrO) (As stated in the Course Structure)

- **Simulate** cache behavior with **the given** mapping method using MATLAB

V. Related ADO(s) (As stated in the Course Structure)

- **Follow** industry standards for system evaluation and performance optimization.
- **Function** effectively in teams to develop processor and memory architectures.
- **Function** responsibly in leadership roles to design computing systems.

VI. Minimum Underpinning Theory (Include relevant content & images for this practicum.)

Cache memory is a small, high-speed memory located close to the CPU, used to reduce access time to frequently used data. Since cache size is limited, mapping techniques determine how data from main memory is placed in the cache. The three common techniques are Direct Mapping (one memory block maps to one cache line), Associative Mapping (any memory block can go to any cache line), and Set-Associative Mapping (a hybrid that groups cache lines into sets).

These techniques influence the hit rate and efficiency of the cache. Direct mapping is simple and fast but prone to collisions, while associative mapping is more flexible but slower due to search overhead. Set-associative mapping offers a balance between speed and flexibility. Analyzing these with tools helps students understand how cache design impacts performance and guides the development of faster computing systems.

VII. Algorithm (Include probable photos of experimental set-up.)

Steps to Analyze Cache Behavior (Direct Mapping)

Step 1: **Define** Cache and Initialize

Choose a small cache size (e.g., 4 slots).

Create an array to represent the cache and set all values to empty (e.g., -1).

Step 2: **Prepare** Memory Access Sequence

Create a list of memory block numbers that simulate how a CPU accesses memory.

Example: [0, 4, 1, 5, 0, 4, 2, 6, 3]

Step 3: **Apply** Direct Mapping Formula

Use the formula:

$\text{cache_index} = \text{block_number} \bmod \text{cache_size}$
 This tells you where in the cache the block should go.

Step 4: Check for Cache Hit or Miss

If the block is already at the calculated cache index → Hit

If not → Miss, and replace the block at that index.

Step 5: Display Result

For each memory access, display:

The block number

The cache index

Whether it was a hit or a miss

The current state of the cache

Step 6: Repeat for All Memory Accesses

Loop through all the blocks in the sequence.

Count the number of hits and misses to understand cache performance.

VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	Laptop	Intel Core i5/i7 processor, 8GB RAM, Windows OS recommended for 8086 emulator	1 No.
2	MATLAB	https://matlab.mathworks.com/	1 No.

IX. Safety Precautions

- Do not run unfamiliar or unverified scripts found online which may harm your computer.
- Verify your register usage and memory addresses to avoid system-level crashes or infinite loops.
- Close all other programs before executing large or unknown assembly code to free up resources.
- Use licensed version of MATLAB to avoid security or legal issues.

X. Procedure (Self-Instructional)

- Switch on the computer.
- Open MATLAB on your computer.
- Create a new script file (File → New → Script).
- Run the following MATLAB script and note the values on the console.
`% cache_simulation_direct.m - Analyze cache behavior using Direct Mapping`

`% Define cache size (number of cache blocks)`
`cacheSize = 4; % For simplicity`
`cache = -1 * ones(1, cacheSize); % Initialize cache with -1 (empty)`

`% Define a sample memory block access pattern`
`memoryBlocks = [0, 4, 1, 5, 0, 4, 2, 6, 3]; % Blocks being accessed`

`% Display headers`
`fprintf('Access\tCache Index\tHit/Miss\tCache State\n');`

`% Loop through each memory block access`
`for i = 1:length(memoryBlocks)`
`block = memoryBlocks(i);`
`index = mod(block, cacheSize); % Direct mapping rule`
`index = index + 1; % MATLAB arrays start at 1`

```

if cache(index) == block
    result = 'Hit';
else
    result = 'Miss';
    cache(index) = block; % Replace with new block
end

% Display result
fprintf('%d\t\t%d\t\t%s\t\t', block, index - 1, result);
disp(cache);
end
5. Shutdown the computer

```

Name	Value	Size	Class
block	3	1×1	double
cache	[4,5,6,3]	1×4	double
cacheSize	4	1×1	double
i	9	1×1	double
index	4	1×1	double
memoryBlo...	[0,4,1,5,0,4,2,6,3]	1×9	double
result	'Miss'	1×4	char

Access	Cache	Index	Hit/Miss	Cache State
0		0	Miss	0 -1 -1 -1
4		0	Miss	4 -1 -1 -1
1		1	Miss	4 1 -1 -1
5		1	Miss	4 5 -1 -1
0		0	Miss	0 5 -1 -1
4		0	Miss	4 5 -1 -1
2		2	Miss	4 5 2 -1
6		2	Miss	4 5 6 -1
3		3	Miss	4 5 6 3

ADDITION QUESTION:

1. Explain how cache improves computer speed in real-world applications like gaming.

CODE:

```

clc; clear;
close all;

cacheSize = 4;
cache = -1 * ones(1, cacheSize);

memoryBlocks = [1 2 3 4 1 2 5 1 2 3 4 5];
hits = 0;
misses = 0;

fprintf('Block\tHit/Miss\tCache State\n');

```

```

for i = 1:length(memoryBlocks)
    block = memoryBlocks(i);

    if ismember(block, cache)
        result = 'HIT';
        hits = hits + 1;

    else
        result = 'MISS';
        misses = misses + 1;
        cache(1:end-1) = cache(2:end);
        cache(end) = block;
    end
    fprintf('%d\t%d\t%s\t\t', block, result);
    disp(cache);

end
fprintf('\nTotal Hits: %d\n', hits);
fprintf('Total Misses: %d\n', misses);

```

block	5	1×1	double
cache	[2,3,4,5]	1×4	double
cacheSize	4	1×1	double
hits	2	1×1	double
i	12	1×1	double
memoryB...	1×12 double	1×12	double
misses	10	1×1	double
result	'MISS'	1×4	char

Block	Hit/Miss	Cache State			
1	MISS	-1	-1	-1	1
2	MISS	-1	-1	1	2
3	MISS	-1	1	2	3
4	MISS	1	2	3	4
1	HIT	1	2	3	4
2	HIT	1	2	3	4
5	MISS	2	3	4	5
1	MISS	3	4	5	1
2	MISS	4	5	1	2
3	MISS	5	1	2	3
4	MISS	1	2	3	4
5	MISS	2	3	4	5

2. Assess how frequent cache misses could affect video streaming or online classes.

CODE:

```

clc; clear;
close all;

cacheSize = 3;
cache = -1 * ones(1, cacheSize);

dataStream = [1 2 3 4 1 2 5 1 2 3 4 5];
hits = 0;
misses = 0;

fprintf('Data\tHit/Miss\tCache State\n');
for i = 1:length(dataStream)
    block = dataStream(i);

    if ismember(block, cache)
        result = 'HIT';
        hits = hits + 1;
    else
        result = 'MISS';
        misses = misses + 1;

        cache(1:end-1) = cache(2:end);
        cache(end) = block;
    end
    fprintf('%d\t%s\t\t', block, result);
    disp(cache);

end
fprintf('\nTotal Cache Hits: %d\n', hits);
fprintf('Total Cache Misses: %d\n', misses);

missRate = (misses/length(dataStream))*100;
fprintf('Cache Miss Rate: %.2f%%\n', missRate);

```

block	5	1×1	double
cache	[5,3,4]	1×3	double
cacheSize	3	1×1	double
dataStream	1×12 double	1×12	double
hits	3	1×1	double
i	12	1×1	double
misses	9	1×1	double
missRate	75	1×1	double
result	'HIT'	1×3	char

Data	Hit/Miss	Cache State		
1	MISS	-1	-1	1
2	MISS	-1	1	2
3	MISS	1	2	3
4	MISS	2	3	4
1	MISS	3	4	1
2	MISS	4	1	2
5	MISS	1	2	5
1	HIT	1	2	5
2	HIT	1	2	5
3	MISS	2	5	3
4	MISS	5	3	4
5	HIT	5	3	4

XI. Actual Resources Used

S. No.	Name of Resource	Broad Specifications		Data	Remarks (If any)
		Tool	Version		
1.	Laptop	Intel Core i5 Processor, 8GB RAM	MATLAB	R2023a	Used to run cache simulation
2.	MATLAB Software	Programming and simulation environment	MATLAB	R2023a	Used to execute the script
3.	Internet	Broadband connection	Chrome Browser	Latest	Used for reference
4.	Operating System	Windows OS	Windows	Windows 11	System environment

XII. Actual Procedure Followed

1. The computer system was switched on and MATLAB software was opened.
2. The provided MATLAB code for cache direct mapping simulation was typed in the script editor.
3. The cache size and memory block access sequence were defined in the program.
4. The direct mapping formula ($\text{block mod cache size}$) was applied in the code.
5. The program was executed using the Run option in MATLAB.
6. The output showing cache index, hit/miss, and cache state was observed in the command window.

XIII. Observations and Dimensional Measurement

S. No.	Observations	Data	Remarks
1	Cache Size	4 blocks	Defined in program
2	Memory Block Sequence	0, 4, 1, 5, 0, 4, 2, 6, 3	Used for simulation
3	Cache Hits	2	When requested block already in cache
4	Cache Misses	7	When block not found in cache

XIV. Results/Observations

1. The MATLAB program successfully simulated direct cache mapping technique.
2. The output displayed the cache index, hit or miss status, and cache state for each memory access.
3. It was observed that direct mapping is simple and fast but causes collisions when multiple blocks map to the same cache line.

XV. Interpretation of Results

The results show how cache memory improves system performance by reducing the time required to access frequently used data. When a requested memory block is already present in the cache, a **cache hit** occurs, resulting in faster data access. When the block is not present, a **cache miss** occurs and the block must be fetched from main memory. Direct mapping is efficient but may lead to conflicts when multiple memory blocks map to the same cache location.

XVI. Conclusions

The practicum successfully demonstrated the **working of direct cache mapping using MATLAB**. The simulation helped in understanding how memory blocks are mapped to cache lines using a simple formula. It also showed the occurrence of cache hits and misses during memory access. From the experiment, it can be concluded that cache mapping techniques play an important role in improving the performance and efficiency of computer systems.

XVII. Suggested Practicum Related Questions:

Note: Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.

1. **Explain** how cache improves computer speed in real-world applications like gaming.
2. **Predict** what happens if cache memory is too small when running multiple programs.
3. **Justify** why industries use different mapping methods (direct, associative, set-associative).
4. **Assess** how frequent cache misses could affect video streaming or online classes.
5. **Determine** which mapping method is more efficient for multitasking systems and explain why.
6. **Examine** what happens if two memory blocks map to the same cache line in direct mapping.
7. **Illustrate** how programmers benefit when cache mapping reduces data access time.
8. **Evaluate** why cache memory is more expensive than RAM yet still necessary in systems.
9. **Discuss** how poor cache performance could slow down banking or airline booking systems.
10. **Propose** how cache mapping techniques can influence the design of modern processors.

XVIII. References / Suggestions for further reading

- a) Cache Mapping Techniques - <https://www.geeksforgeeks.org/computer-organization-architecture/cache-mapping-techniques/>
- b) Cache Mapping and Types = <https://youtu.be/hhLdy3J9oqg>

XIX. Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the 'process-related skills/LOs' (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and 'product-related skills/LOs'

Note: The weightage for the **process-related skills** and **product-related skills** will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	<u>Follow</u> industry standards	3	

Performance Indicators		Maximum Marks	Marks Obtained
2	Function effectively in teams	2	
3	Function responsibly in leadership	3	
4	Implement cache mapping techniques and simulate direct memory access operations	10	
Product-related Skills: 12 Marks - 40%			
5	Uses correct commands or syntax in Tool	3	
6	Results	3	
7	Interpretation of Results	3	
8	Conclusion	3	
Total		30	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	Neavis Rishva. J	URK25CS1236

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	Product-Related Skills (12)	Total (30)		
